

IBM AI RACING LEAGUE

Università degli Studi di Salerno · IBM SkillsBuild · Codemotion

Guida autonoma in TORCS

mediante **Behavioral Cloning**

Diagnosi, controllo manuale e raccolta dati per l'addestramento di un agente di guida

 Circuito: Corkscrew

 Driver: scr_server

 A.A. 2025 / 2026

GRUPPO 8

Santoriello Pasquale · Fantis Aya · Crisci Antonio · Rossomando Andrea · De Cicco Martina

L'obiettivo della competizione

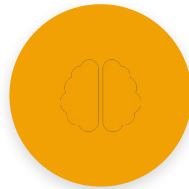
Sviluppare un agente di intelligenza artificiale capace di guidare in **autonomia** un'auto nel simulatore **TORCS**, percorrendo il circuito Corkscrew nel minor tempo possibile, senza uscire di pista e senza perdere aderenza.

Il progetto si articola in tre fasi logiche



1 · Raccolta dati

Un operatore umano guida manualmente, registrando a ogni istante lo stato dei sensori e l'azione di controllo corrispondente.



2 · Addestramento

Un modello impara ad associare ogni stato sensoriale alla relativa azione di guida, a partire dai dati raccolti.



3 · Guida autonoma

Il modello addestrato sostituisce il pilota umano e controlla il veicolo in completa autonomia.

TORCS e i moduli Python

Il veicolo è pilotato dal driver **scr_server**, che espone i sensori e accetta i comandi tramite connessione di rete (socket UDP).



manual_control_ps4_1.py

Guida manuale con controller PS4 (libreria pygame): traduce i tasti in sterzo, accelerazione e freno e registra ogni azione nei file CSV.



training_modello.py

Acquisisce i giri di guida manuale (CSV) e li trasforma nei pesi della rete neurale capace di guidare in autonomia.



guida_bc_nc_1.py

Abilita la guida autonoma nel simulatore tramite il modello di Behavioral Cloning, replicando traiettoria e stile appresi.

Input del modello e azioni di controllo

FEATURE DI INGRESSO (22 dimensioni)

Grandezza	Tipo	Significato
speedX	sensore	Velocità longitudinale (km/h)
angle	sensore	Angolo del veicolo rispetto all'asse pista
trackPos	sensore	Posizione laterale rispetto al centro pista
track[0...18]	sensore	19 sensori telemetrici: distanza dal bordo pista
rpm	sensore	Numero di giri al minuto del motore
damage	sensore	Danno attuale dell'auto

COMANDI DI USCITA

steer $[-1, 1]$

Sterzo — Levetta analogica sinistra

accel $[0, 1]$

Accelerazione — Grilletto progressivo R2

brake $[0, 1]$

Freno — Grilletto progressivo L2

Perché Behavioral Cloning e Rete Neurale



Behavioral Cloning

Approccio supervisionato semplice, intuitivo e veloce: il modello impara osservando i dati di un pilota umano.

vs Reinforcement Learning, che richiede innumerevoli tentativi casuali.



Rete Neurale

Racchiude la conoscenza nei pesi: a runtime esegue solo veloci moltiplicazioni, con risposta rapidissima anche su dataset enormi.

Più dati → più precisa e robusta.



Preferita al k-NN

Il k-NN diventa sempre più lento e pesante all'aumentare dei dati raccolti.

La rete mantiene inferenza costante e leggera.

Raccolta dati di guida manuale

Tramite `manual_control_ps4_1.py` e il controller PS4 abbiamo registrato i nostri giri, ampliando progressivamente il dataset con situazioni sempre più diverse.



giri completati

**48.453**

campioni raccolti

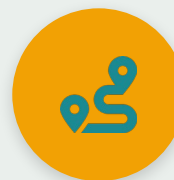
**10 Hz**

frequenza (≈ 0.1 s)



Traiettorie ideali

All'inizio: macchina sempre al centro della pista, puntando a un ottimo tempo.

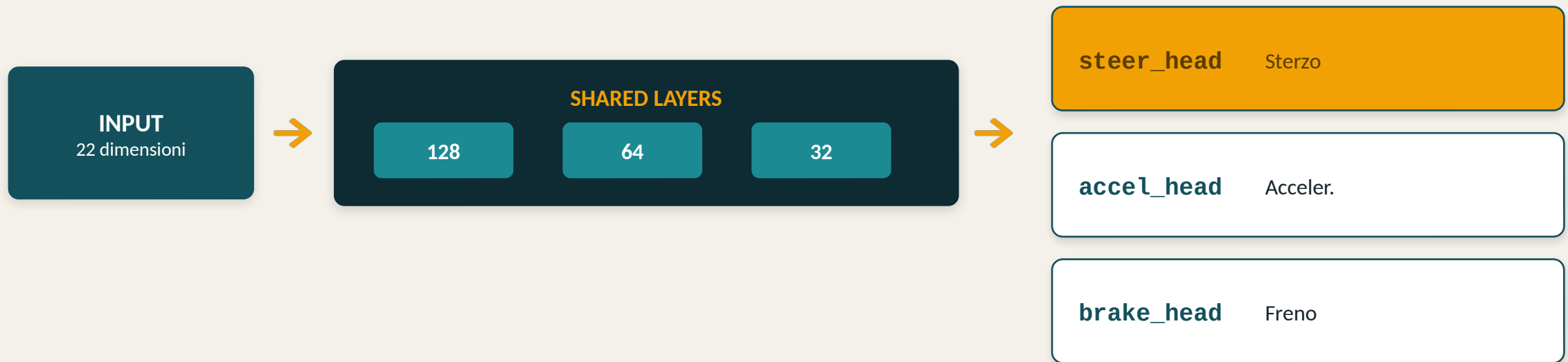


Giri imperfetti voluti

Veicolo portato sui bordi, per insegnare all'agente a correggere lo sterzo e rientrare al centro.

TorcsNet — Multi-Task Learning

Pre-processing: pulizia dati (rimozione step a velocità nulla, fuori pista, danno elevato), normalizzazione con StandardScaler, split 80% / 20%.



Branching Heads — 3 output indipendenti

Spina dorsale condivisa di 3 layer densi (128-64-32) con attivazioni ReLU per estrarre le feature spaziali e cinematiche, poi diramata in 3 teste dedicate. Framework: **PyTorch**.

Vincoli fisici e Steer-Weighted Loss



Vincoli fisici (funzioni di attivazione)

Tanh → sterzo

limita l'uscita in $[-1, 1]$, come un volante.

Sigmoid → pedali

vincola accel. e freno in $[0, 1]$: mai comandi impossibili.

Impostazioni di addestramento

Ottimizzatore	Adam · learning rate 0.001
Hardware	Addestramento su GPU
Regolarizzazione	Dropout $p = 0.1$ (anti-overfitting)

STEER-WEIGHTED LOSS

Sbagliare lo sterzo è più grave che sbagliare l'acceleratore: una traiettoria errata porta subito fuori pista.

$$L_{total} = 5 \cdot L_{steer} + 1 \cdot L_{accel} + 2 \cdot L_{brake}$$

5.0

steer

1.0

accel

2.0

brake

Pesi scelti sperimentalmente: priorità alla correttezza della direzione, per maggiore stabilità in curva.

Il ciclo di guida in tempo reale

guida_bc_nc_1.py carica modello_torcs_pytorch.pth e scaler_torcs.pkl, poi si connette a TORCS con la libreria SnakeOil.



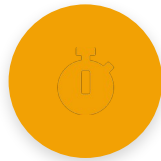
Le marce non sono apprese dalla rete: la marcia ottimale è selezionata automaticamente in base alla velocità corrente.

Una guida autonoma fluida e sicura



**Giro
completato**

in autonomia



1:13

miglior giro manuale (dataset)



1:45

tempo finale del modello



Conclusioni

L'auto guida in modo autonomo: il modello ha appreso a coordinare **sterzo, acceleratore e freno** senza alcun intervento esterno, completando il percorso in modo fluido e sicuro. Il Behavioral Cloning ha garantito **stabilità dinamica** ed evitato danni, imitando con successo le traiettorie del dataset.